



University of Tehran
College of Engineering
School of Electrical and Computer
Engineering



Machine Learning

Assignment 1

Full Name
Sepanta Ghonoodi

Student ID
810102483

November 8, 2025

Contents

1	Question 1: Bayes Rule vs. Random Decisions	2
1.1	Overall Risk for Randomized Rule	2
1.2	Comparison of Risks	3
1.3	Conditions for Equivalence	4
2	Question 2: Bayes Boundaries for Gaussian Classes	5
2.1	Single Feature Case	5
2.2	Two Feature Analysis	5
2.2.1	a	6
2.2.2	b	6
2.2.3	c	7
2.2.4	d	7
3	Question 3: Naive Bayes Parameter Estimation (MLE)	8
3.1	Formulating the Joint Log-Likelihood Function	8
3.2	Deriving the MLEs	9
4	Question 4: Modeling Goals with MLE	10
4.1	Deriving the MLE for the Poisson Parameter	10
4.2	Numerical Estimate and Prediction after Five Games	11
4.3	Updated Estimate and Prediction after Six Games	11
5	Question 5: MLE Derivation of the Least Squares Loss	11
5.1	Likelihood Function for a Single Data Point	11
5.2	Equivalence of Log-Likelihood and Mean Squared Error	12
6	Question 6: Effect of Priors on the Decision Boundary	13
6.1	Deriving the Decision Boundary Condition	13
7	Question 7: Naive Bayes Warm-Up	14
7.1	Part 1	14
7.1.1	Manual Derivation	14
7.1.2	Python Implementation	15
7.1.3	Program Output	15
7.2	Part 2	15

Abstract

This assignment explores the fundamentals of Bayesian Decision Theory and probabilistic modeling. It features a comparative risk analysis for different decision-making strategies, contrasting the optimal Bayes classifier with a randomized rule. The assignment also focuses on deriving Bayesian decision boundaries for classifiers under both univariate and multivariate Gaussian assumptions. Furthermore, Maximum Likelihood Estimation (MLE) is applied to estimate parameters for Bernoulli (Naive Bayes) and Poisson distributions, demonstrating that the resulting estimates are equivalent to the data's empirical frequencies. Finally, the analysis formally connects the probabilistic approach of MLE for linear regression with the common optimization objective of minimizing Mean Squared Error (MSE).

Question 1: Bayes Rule vs. Random Decisions

Suppose the task is to classify the input signal x into one of K classes $\omega \in \{1, 2, \dots, K\}$ such that the action $\alpha(x) = i$ means classifying x into class i . The Bayesian decision rule is to maximize the posterior probability

$$\alpha_{Bayes}(x) = \omega^* = \arg \max_i p(\omega_i|x).$$

Suppose we replace it by a randomized decision rule, which classifies x to class i following the posterior probability $p(\omega_i|x)$, i.e.,

$$\alpha_{rand}(x) = \omega \sim p(\omega|x).$$

1.1 Overall Risk for Randomized Rule

What is the overall risk R_{rand} for this decision rule? Derive it in terms of the posterior probability using the zero-one loss function.

we know

$$R(\alpha_{rand}|x) = \sum_i \lambda(\alpha_{rand}|\omega_i) P(\omega_i|x)$$

and we know that

$$\lambda(\alpha_{rand}|\omega_i) = \begin{cases} 0 & \alpha_{rand}(x) = \omega_i \\ 1 & \alpha_{rand}(x) \neq \omega_i \end{cases}$$

$\Rightarrow$

$$\begin{aligned} \lambda(\alpha_{rand}|\omega_i) &= 1 \cdot P(\alpha_{rand}(x) \neq \omega_i) + 0 \cdot P(\alpha_{rand}(x) = \omega_i) \\ &= 1 - P(\alpha_{rand}(x) = \omega_i) = 1 - P(\omega_i|x) \end{aligned}$$

$\Rightarrow$

$$\begin{aligned} R(\alpha_{rand}|x) &= \sum_i (1 - P(\omega_i|x)) P(\omega_i|x) \\ &= \underbrace{\sum_i P(\omega_i|x)}_1 - \sum_i P^2(\omega_i|x) \\ &= 1 - \sum_{i=1}^k P^2(\omega_i|x) \end{aligned}$$

Finally, the overall risk R_{rand} is the expectation of this conditional risk:

$$\begin{aligned} R_{rand} &= E_x[R(\alpha_{rand}|x)] \\ &= \int \left(1 - \sum_{i=1}^k P^2(\omega_i|x)\right) p(x) dx \\ &= \int p(x) dx - \int \left(\sum_{i=1}^k P^2(\omega_i|x)\right) p(x) dx \end{aligned}$$

and we know

$$\int p(x) dx = 1$$

$\Rightarrow$

$$R_{rand} = 1 - \int \left(\sum_{i=1}^k P^2(\omega_i|x)\right) p(x) dx$$

1.2 Comparison of Risks

2. Show that this risk R_{rand} is always no smaller than the Bayes risk R_{Bayes} . Thus, we cannot benefit from the randomized decision.

$$R(\alpha_{bayes}|x) = \sum_{i=1}^k \lambda(\alpha_{bayes}|\omega_i) P(\omega_i|x)$$

$\Rightarrow$

$$\lambda(\alpha_{bayes}|\omega_i) = \begin{cases} 0 & \alpha_{bayes} = \omega_i \\ 1 & \alpha_{bayes} \neq \omega_i \end{cases}$$

$\Rightarrow$

$$\begin{aligned} R(\alpha_{bayes}|x) &= 1 \times P(\alpha_{bayes} \neq \omega_i) \\ &= 1 - P(\alpha_{bayes} = \omega_i) \\ &= 1 - \max_i P(\omega_i|x) \end{aligned}$$

$$\begin{aligned} R_{bayes} &= E_x[R(\alpha_{bayes}|x)] = \int \left(1 - \max_i P(\omega_i|x)\right) p(x) dx \\ &= 1 - \int \max_i P(\omega_i|x) p(x) dx \end{aligned}$$

we want to prove that

$$1 - \int \left(\sum_{i=1}^k P^2(\omega_i|x)\right) p(x) dx > 1 - \int \max_i P(\omega_i|x) p(x) dx$$

$\Rightarrow$

$$\sum_{i=1}^k P^2(\omega_i|x) < \max_i P(\omega_i|x)$$

we know

$$P(\omega_i|x) \leq \max_i P(\omega_i|x)$$

$\implies$

$$(P(\omega_i|x))^2 \leq \max_i P(\omega_i|x) P(\omega_i|x)$$

sum of all these for $i = 1..k \implies$

$$\sum_{i=1}^k (P(\omega_i|x))^2 \leq \max_i P(\omega_i|x) \underbrace{\sum_{i=1}^k P(\omega_i|x)}_1$$

$\implies$ it proves that the risk for bayesian is better

1.3 Conditions for Equivalence

3. Under what conditions on the posterior are the two decision rules the same?

If one class has $p(\omega_i|x) = 1$ therefore other classes will have $p(\omega_i|x) = 0$

$\implies$ Risk for bayesian rule $\implies$

$$1 - \max_i P(\omega_i|x) = 1 - 1 = 0$$

Risk for random $\implies$

$$1 - \sum_{i=1}^k P^2(\omega_i|x) = 1 - (1^2 + 0 + \dots + 0) = 0$$

therefore they will be equal.

if all classes have the same probability, assuming the bayesian will take one class randomly. $\implies$
 $p(\omega_i|x) = 1/k$

Risk for bayesian

$$1 - \max_i P(\omega_i|x) = 1 - P(\omega_i|x) = 1 - 1/k$$

Risk for random

$$1 - \sum_{i=1}^k P^2(\omega_i|x) = 1 - k \left(\frac{1}{k^2} \right) = 1 - \frac{1}{k}$$

$\implies$ in these two scenarios they will have equal risk

Question 2: Bayes Boundaries for Gaussian Classes

Suppose we have a two-class recognition problem with salmon ($\omega = 1$) and sea bass ($\omega = 2$).

2.1 Single Feature Case

1. First, assume we have one feature, the pdfs are the Gaussians $N(0, \sigma^2)$ and $N(1, \sigma^2)$ for the two classes, respectively. Show that the threshold τ minimizing the average risk is equal to

$$\tau = \frac{1}{2} - \sigma^2 \ln \frac{\lambda_{12} P(\omega_2)}{\lambda_{21} P(\omega_1)}$$

where we have assumed $\lambda_{11} = \lambda_{22} = 0$.

bayes decision rule

$$\frac{P(x|\omega_1)P(\omega_1)}{P(x|\omega_2)P(\omega_2)} > \frac{\lambda_{12}}{\lambda_{21}}$$

$\Rightarrow$

$$\begin{aligned} \frac{\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{x^2}{2\sigma^2}} P(\omega_1)}{\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-1)^2}{2\sigma^2}} P(\omega_2)} &= \frac{\lambda_{12}}{\lambda_{21}} \\ e^{-\frac{x^2}{2\sigma^2}} P(\omega_1) \lambda_{21} &= e^{-\frac{(x-1)^2}{2\sigma^2}} P(\omega_2) \lambda_{12} \end{aligned}$$

$\ln \Rightarrow$

$$-\frac{x^2}{2\sigma^2} + \ln(P(\omega_1)) + \ln(\lambda_{21}) = -\frac{(x-1)^2}{2\sigma^2} + \ln(P(\omega_2)) + \ln(\lambda_{12})$$

$$\frac{1}{2} \frac{(x-1)^2 - x^2}{\sigma^2} = \ln(P(\omega_2)) - \ln(P(\omega_1)) + \ln(\lambda_{12}) - \ln(\lambda_{21})$$

$\Rightarrow$

$$\frac{1}{\sigma^2} \left(-x + \frac{1}{2} \right) = \ln \left(\frac{P(\omega_2) \cdot \lambda_{12}}{P(\omega_1) \cdot \lambda_{21}} \right)$$

$\Rightarrow$

$$x = \frac{1}{2} - \sigma^2 \ln \left(\frac{\lambda_{12} P(\omega_2)}{\lambda_{21} P(\omega_1)} \right)$$

2.2 Two Feature Analysis

Next, suppose we have two features $\mathbf{x} = (x_1, x_2)$ and the two class-conditional densities, $p(\mathbf{x}|\omega = 1)$ and $p(\mathbf{x}|\omega = 2)$, are 2D Gaussian distributions centered at points $(4, 11)$ and $(10, 3)$ respectively with the

same covariance matrix $\Sigma = 3I$ (where I is the identity matrix). Suppose the priors are $P(\omega = 1) = 0.6$ and $P(\omega = 2) = 0.4$.

2.2.1 a

Suppose we use a Bayes decision rule, write the two discriminant functions $g_1(\mathbf{x})$ and $g_2(\mathbf{x})$.

$$g_i(x) = P(x|\omega_i)P(\omega_i) = \frac{1}{(2\pi)^{|\Sigma|^{1/2}}} e^{-\frac{1}{2}(x-\mu_i)^T \Sigma^{-1}(x-\mu_i)} \times P(\omega_i)$$

$\Rightarrow$

$$\ln(g_i(x)) = \ln(2\pi) - \frac{1}{2} \ln(|\Sigma|) - \frac{1}{2}(x - \mu_i)^T \Sigma^{-1}(x - \mu_i) + \ln(P(\omega_i))$$

Since $\ln(2\pi) - \frac{1}{2} \ln(|\Sigma|)$ appears in both functions, we ignore them

$$g_1(x) = -\frac{1}{2} \left(x - \begin{pmatrix} 4 \\ 11 \end{pmatrix} \right)^T (3I)^{-1} \left(x - \begin{pmatrix} 4 \\ 11 \end{pmatrix} \right) + \ln(0.6)$$

$$g_2(x) = -\frac{1}{2} \left(x - \begin{pmatrix} 10 \\ 3 \end{pmatrix} \right)^T (3I)^{-1} \left(x - \begin{pmatrix} 10 \\ 3 \end{pmatrix} \right) + \ln(0.4)$$

2.2.2 b

Derive the equation for the decision boundary $g_1(\mathbf{x}) = g_2(\mathbf{x})$.

decision boundary : (Set $g_1(x) = g_2(x)$ and simplify)

$$-\frac{1}{2}(x - \mu_1)^T \Sigma^{-1}(x - \mu_1) + \ln(P(\omega_1)) = -\frac{1}{2}(x - \mu_2)^T \Sigma^{-1}(x - \mu_2) + \ln(P(\omega_2))$$

$$\frac{1}{2} [(x - \mu_2)^T \Sigma^{-1}(x - \mu_2) - (x - \mu_1)^T \Sigma^{-1}(x - \mu_1)] = \ln(P(\omega_2)) - \ln(P(\omega_1))$$

(Substitute $\Sigma^{-1} = \frac{1}{3}I$, $\mu_1 = \begin{pmatrix} 4 \\ 11 \end{pmatrix}$, $\mu_2 = \begin{pmatrix} 10 \\ 3 \end{pmatrix}$, $x = \begin{pmatrix} x_1 \\ x_2 \end{pmatrix}$)

$$\frac{1}{2} \cdot \frac{1}{3} ((x_1 - 10)^2 + (x_2 - 3)^2 - [(x_1 - 4)^2 + (x_2 - 11)^2]) = \ln\left(\frac{0.4}{0.6}\right)$$

$\Rightarrow$

$$\frac{1}{6} ((x_1^2 - 20x_1 + 100 + x_2^2 - 6x_2 + 9) - (x_1^2 - 8x_1 + 16 + x_2^2 - 22x_2 + 121)) = \ln\left(\frac{0.4}{0.6}\right)$$

$\Rightarrow$

$$\frac{1}{6} (-12x_1 + 16x_2 - 28) = \ln\left(\frac{0.4}{0.6}\right)$$

$\Rightarrow$

$$-12x_1 + 16x_2 = 6 \ln\left(\frac{0.4}{0.6}\right) + 28 \rightarrow \text{decision boundary}$$

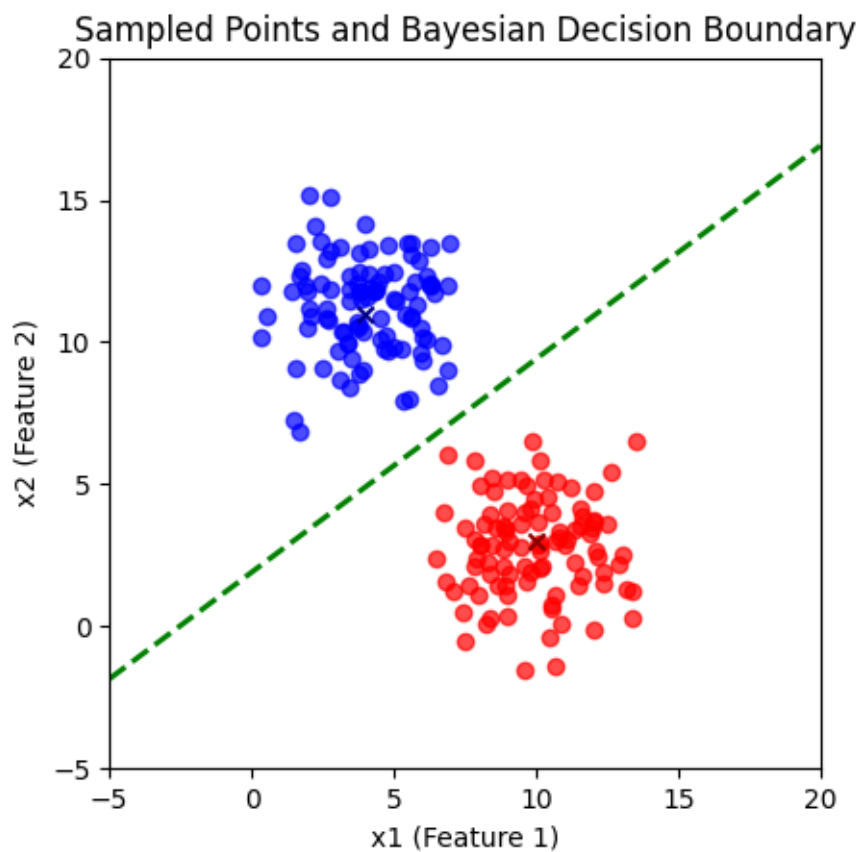
2.2.3 c

How would the decision boundary change if we changed the priors? the covariances?

based on the derived equation, changing the prior will only change the constant term and therefore shifts the decision boundary, but if we change covariance matrix, it will affect the slope of the boundary.

2.2.4 d

Using computer software, sample 100 points from each of the two densities. Draw the boundary on the feature space (the 2D plane).



Question 3: Naive Bayes Parameter Estimation (MLE)

The goal is to find the Maximum Likelihood Estimates (MLEs) for the parameters $\phi = \{\phi_y, \phi_{j|y=0}, \phi_{j|y=1}\}$ by maximizing the joint log-likelihood $\ell(\phi)$. Model Parameters for Bernoulli Distribution are:

- Prior Probability:

$$\phi_y = P(y = 1)$$

- Feature Likelihoods:

$$\phi_{j|y=0} = P(x_j = 1|y = 0)$$

$$\phi_{j|y=1} = P(x_j = 1|y = 1)$$

3.1 Formulating the Joint Log-Likelihood Function

Write the Joint Log-Likelihood Function $\ell(\phi)$.

$$L(\phi) = P(D|\phi) = \prod_i P(x^{(i)}, y^{(i)}|\phi)$$

$\Rightarrow$

$$\ell(\phi) = \sum_i (\ln P(x^i|y^i, \phi) + \ln P(y_i|\phi))$$

$$= \sum_i \ln P(x^i|y^i, \phi) + \sum_i \ln P(y_i|\phi) \quad (I)$$

Based on Bernoulli distribution:

$$P(y_i|\phi_y) = (\phi_y)^{y_i} (1 - \phi_y)^{1-y_i}$$

$$P(x_j|y = 0) = (\phi_{j|y=0})^{x_j} (1 - \phi_{j|y=0})^{1-x_j}$$

$$P(x_j|y = 1) = (\phi_{j|y=1})^{x_j} (1 - \phi_{j|y=1})^{1-x_j}$$

$\Rightarrow$ using this in (I)

$\Rightarrow$

$$P(x_j|y_i) = ((\phi_{j|y=1})^{x_j} (1 - \phi_{j|y=1})^{1-x_j})^{y_i} \times ((\phi_{j|y=0})^{x_j} (1 - \phi_{j|y=0})^{1-x_j})^{1-y_i}$$

$\Rightarrow$

$$P(x_i|y_i) = \prod_{j=1}^m \left[((\phi_{j|y=1})^{x_j} (1 - \phi_{j|y=1})^{1-x_j})^{y_i} \times ((\phi_{j|y=0})^{x_j} (1 - \phi_{j|y=0})^{1-x_j})^{1-y_i} \right]$$

$\Rightarrow$

$$\begin{aligned} \ell(\phi) = \sum_i \left\{ y_i \sum_{j=1}^m [x_j \ln(\phi_{j|y=1}) + (1 - x_j) \ln(1 - \phi_{j|y=1})] \right. \\ \left. + (1 - y_i) \sum_{j=1}^m [x_j \ln(\phi_{j|y=0}) + (1 - x_j) \ln(1 - \phi_{j|y=0})] \right. \\ \left. + y^{(i)} \ln(\phi_y) + (1 - y^{(i)}) \ln(1 - \phi_y) \right\} \end{aligned}$$

3.2 Deriving the MLEs

Show that the MLEs correspond to the empirical frequencies. for $\phi_y \Rightarrow$

$$\frac{\partial \ell(\phi)}{\partial \phi_y} = \sum_i \left(\frac{y^i}{\phi_y} - \frac{1 - y^i}{1 - \phi_y} \right) = 0$$

$\Rightarrow$

$$\frac{1}{\phi_y} \sum_i y_i = \frac{1}{1 - \phi_y} \left(m - \sum_i y_i \right)$$

$\Rightarrow$

$$\left(\frac{1}{\phi_y} + \frac{1}{1 - \phi_y} \right) \sum_i y_i = \frac{m}{1 - \phi_y}$$

$$\frac{1}{\phi_y(1 - \phi_y)} \sum_i y_i = \frac{m}{1 - \phi_y}$$

$\Rightarrow$

$$\phi_y = \frac{\sum_i y_i}{m}$$

ϕ_y is the total number of samples labeled 1, divided by the total number of samples.

for $\phi_{j|y=1}$

$$\frac{\partial \ell(\phi)}{\partial \phi_{j|y=1}} = \sum_{i=1}^m y^{(i)} \left[\frac{x_j^{(i)}}{\phi_{j|y=1}} - \frac{1 - x_j^{(i)}}{1 - \phi_{j|y=1}} \right] = 0$$

$\Rightarrow$

$$\sum_{i=1}^m y^{(i)} x_j^{(i)} (1 - \phi_{j|y=1}) = \sum_{i=1}^m y^{(i)} (1 - x_j^{(i)}) \phi_{j|y=1}$$

$\Rightarrow$

$$\phi_{j|y=1} = \frac{\sum_{i=1}^m y^{(i)} x_j^{(i)}}{\sum_{i=1}^m y^{(i)}}$$

$\phi_{j|y=1}$ is the total number of samples labeled 1 that also have feature $j = 1$, divided by the total number of samples labeled 1.

for $\phi_{j|y=0}$

$$\frac{\partial \ell(\phi)}{\partial \phi_{j|y=0}} = \sum_i (1 - y^{(i)}) \left(\frac{x_j^{(i)}}{\phi_{j|y=0}} - \frac{1 - x_j^{(i)}}{1 - \phi_{j|y=0}} \right) = 0$$

$$\sum_i (1 - y^{(i)}) x_j^{(i)} (1 - \phi_{j|y=0}) = \sum_{i=1}^m (1 - y^{(i)}) (1 - x_j^{(i)}) \phi_{j|y=0}$$

$$\phi_{j|y=0} = \frac{\sum_{i=1}^m (1 - y^{(i)}) x_j^{(i)}}{\sum_{i=1}^m (1 - y^{(i)})}$$

$\phi_{j|y=0}$ is the total number of samples labeled 0 that also have feature $j = 1$, divided by the total number of samples labeled 0.

Question 4: Modeling Goals with MLE

You are the Reign FC manager, and the team is five games into its 2021 season. The number of goals scored by the team in each game so far are given below:

$$[2, 4, 6, 0, 1].$$

Let's call these scores $x_1, \dots, x_5$. Based on your (assumed iid) data, you'd like to build a model to understand how many goals the Reign are likely to score in their next game. You decide to model the number of goals scored per game using a Poisson distribution. Recall that the Poisson distribution with parameter λ assigns every non-negative integer $x = 0, 1, 2, \dots$ a probability given by

$$\text{Poi}(x|\lambda) = \frac{e^{-\lambda} \lambda^x}{x!}$$

4.1 Deriving the MLE for the Poisson Parameter

Derive an expression for the maximum-likelihood estimate of the parameter λ governing the Poisson distribution in terms of goal counts for the first n games: $x_1, \dots, x_n$. (Hint: remember that the log of the likelihood has the same maximizer as the likelihood function itself.)

$$L(\lambda) = \prod_{i=1}^n \text{Poi}(x_i|\lambda) = \prod_{i=1}^n \frac{e^{-\lambda} \lambda^{x_i}}{x_i!}$$

$$\ell(\lambda) = \sum_{i=1}^n \ln(\text{Poi}(x_i|\lambda)) = \sum_{i=1}^n (-\lambda + x_i \ln(\lambda) - \ln(x_i!))$$

$$\implies \frac{\partial}{\partial \lambda} \implies$$

$$-n + \sum_{i=1}^n \frac{x_i}{\lambda} = 0 \implies \lambda = \frac{1}{n} \sum_{i=1}^n x_i$$

$\Rightarrow$ the poisson parameter is the mean of number of goals in games.

4.2 Numerical Estimate and Prediction after Five Games

Give a numerical estimate of λ after the first five games. Given this λ , what is the probability that the Reign score 6 goals in their next game?

$$\lambda = \frac{1}{n} \sum_{i=1}^n x_i \Rightarrow \lambda = \frac{2+4+6+1}{5} = \frac{13}{5} = 2.6$$

$\Rightarrow$

$$\text{Poi}(6|\lambda = 2.6) = \frac{e^{-2.6} \times (2.6)^6}{6!} \approx 0.031$$

4.3 Updated Estimate and Prediction after Six Games

Suppose the Reign score 8 goals in their 6th game. Give an updated numerical estimate of λ after six games and compute the probability that the Reign score 6 goals in their 7th game.

$$\lambda = \frac{2+4+6+1+8}{6} = \frac{21}{6} = \frac{7}{2} = 3.5$$

$$\text{Poi}(6|\lambda = 3.5) = \frac{e^{-3.5}(3.5)^6}{6!} \approx 0.077$$

As you can see by scoring more goals in one game, the probability of scoring 6 goals in the subsequent games increased as expected.

Question 5: MLE Derivation of the Least Squares Loss

In **Linear Regression**, we assume the relationship between input $\mathbf{x}^{(i)}$ and output $y^{(i)}$ is $y^{(i)} = \boldsymbol{\theta}^T \mathbf{x}^{(i)} + \epsilon^{(i)}$, where the error $\epsilon^{(i)}$ is independent and follows a **Gaussian (Normal) distribution** with mean zero and variance σ^2 : $\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$.

5.1 Likelihood Function for a Single Data Point

Write the likelihood $p(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}, \sigma^2)$.

since $y^{(i)} = \boldsymbol{\theta}^T \mathbf{x}^{(i)} + \epsilon^i$ and ϵ^i follows gaussian distribution and $\boldsymbol{\theta}^T \mathbf{x}^{(i)}$ is constant therefore $y^{(i)}$ follows a gaussian distribution so we should only find mean and variance of this distribution

$$\begin{aligned}
E[y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}, \sigma^2] &= E[\boldsymbol{\theta}^T \mathbf{x}^{(i)} + \epsilon^{(i)}] \\
&= \underbrace{E[\boldsymbol{\theta}^T \mathbf{x}^{(i)}]}_{\text{Const.}} + \underbrace{E[\epsilon^{(i)}]}_0 = \boldsymbol{\theta}^T \mathbf{x}^{(i)}
\end{aligned}$$

$$\text{Var}(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}, \sigma^2) = \text{Var}(\boldsymbol{\theta}^T \mathbf{x}^{(i)} + \epsilon^{(i)})$$

Constant term doesn't change variance

$$= \text{Var}(\epsilon^{(i)}) = \sigma^2$$

$\Rightarrow$

$$y^{(i)} \in \mathcal{N}(\boldsymbol{\theta}^T \mathbf{x}^{(i)}, \sigma^2)$$

$$P(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \boldsymbol{\theta}^T \mathbf{x}^{(i)})^2}{2\sigma^2}\right)$$

5.2 Equivalence of Log-Likelihood and Mean Squared Error

Show that maximizing the log-likelihood $\ell(\boldsymbol{\theta})$ with respect to $\boldsymbol{\theta}$ is equivalent to minimizing the squared error cost function (Mean Squared Error).

$$P(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}, \sigma^2) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \boldsymbol{\theta}^T \mathbf{x}^{(i)})^2}{2\sigma^2}\right)$$

$$L(\boldsymbol{\theta}) = \prod_i P(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}, \sigma^2)$$

$$\ell(\boldsymbol{\theta}) = \sum_i \ln(P(y^{(i)}|\mathbf{x}^{(i)}; \boldsymbol{\theta}, \sigma^2))$$

$\Rightarrow$

$$\ell(\boldsymbol{\theta}) = \sum_i \left(-\ln(\sqrt{2\pi}\sigma) - \frac{1}{2\sigma^2} (y^{(i)} - \boldsymbol{\theta}^T \mathbf{x}^{(i)})^2 \right)$$

$\Rightarrow$

$$\boldsymbol{\theta} = \arg \max_{\boldsymbol{\theta}} \sum_i \left(-\ln(\sqrt{2\pi}\sigma) - \frac{1}{2\sigma^2} (y^{(i)} - \boldsymbol{\theta}^T \mathbf{x}^{(i)})^2 \right)$$

(Remove constant terms that don't depend on $\boldsymbol{\theta}$)

$$\arg \max_{\boldsymbol{\theta}} \sum_i \left(-\frac{1}{2\sigma^2} (y^{(i)} - \boldsymbol{\theta}^T \mathbf{x}^{(i)})^2 \right)$$

(Maximizing a negative is minimizing the positive. Remove the constant $-\frac{1}{2\sigma^2}$)

$$\arg \min_{\boldsymbol{\theta}} \sum_i (y^{(i)} - \boldsymbol{\theta}^T \mathbf{x}^{(i)})^2$$

we know that

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n \left(\underbrace{y^{(i)}}_{\text{true value}} - \underbrace{\theta^T \mathbf{x}^{(i)}}_{\text{predicted}} \right)^2$$

therefore minimizing the MSE is equal to maximizing the log likelihood

Question 6: Effect of Priors on the Decision Boundary

Consider a two-class, one-dimensional problem ($\mathbf{x} = x, \omega_1, \omega_2$) where the likelihoods are standard Gaussians with equal variance σ^2 but different means:

$$p(x|\omega_1) \sim \mathcal{N}(\mu_1, \sigma^2) \quad \text{and} \quad p(x|\omega_2) \sim \mathcal{N}(\mu_2, \sigma^2)$$

6.1 Deriving the Decision Boundary Condition

Derive the decision boundary condition (x^*) if the **priors are unequal**: $P(\omega_1)$ and $P(\omega_2)$. Explain how the value of x^* shifts as $P(\omega_1)$ increases relative to $P(\omega_2)$.

$$P(x|\omega_1)P(\omega_1) = P(x|\omega_2)P(\omega_2)$$

$$p(x|\omega_1) \sim \mathcal{N}(\mu_1, \sigma^2) \quad p(x|\omega_2) \sim \mathcal{N}(\mu_2, \sigma^2)$$

$$\frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu_1)^2}{2\sigma^2}\right) P(\omega_1) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x-\mu_2)^2}{2\sigma^2}\right) P(\omega_2)$$

ln $\Rightarrow$

$$-\frac{(x-\mu_1)^2}{2\sigma^2} + \ln(P(\omega_1)) = -\frac{(x-\mu_2)^2}{2\sigma^2} + \ln(P(\omega_2))$$

$\Rightarrow$

$$\frac{1}{2\sigma^2} ((x-\mu_2)^2 - (x-\mu_1)^2) = \ln\left(\frac{P(\omega_2)}{P(\omega_1)}\right)$$

$\Rightarrow$

$$\frac{2(\mu_1 - \mu_2)x + \mu_2^2 - \mu_1^2}{2\sigma^2} = \ln\left(\frac{P(\omega_2)}{P(\omega_1)}\right)$$

$$\frac{2(\mu_1 - \mu_2)}{2\sigma^2}x - \ln\left(\frac{P(\omega_2)}{P(\omega_1)}\right) + \frac{\mu_2^2 - \mu_1^2}{2\sigma^2} = 0$$

$$x = \frac{2\sigma^2 \ln\left(\frac{P(\omega_2)}{P(\omega_1)}\right) - (\mu_2^2 - \mu_1^2)}{2(\mu_1 - \mu_2)}$$

by increasing $P(\omega_1)$ w.r.t $P(\omega_2)$ we decrease $\ln\left(\frac{P(\omega_2)}{P(\omega_1)}\right)$ therefore the decision boundary moves to left and the chance of class 2 being selected increases.

Question 7: Naive Bayes Warm-Up

In this problem, we'll first review a standard way that Bayes rule is used and then explore implementing a Naive Bayes classifier, a very fast classification method that is often surprisingly accurate for text data with simple representations like bag of words.

7.1 Part 1

7.1.1 Manual Derivation

We want to find $P(D = 1|T = 1)$. We use Bayes' rule:

$$P(D = 1|T = 1) = \frac{P(T = 1|D = 1)P(D = 1)}{P(T = 1)}$$

We are given:

- $P(T = 1|D = 1) = 0.99$ (True Positive)
- $P(D = 1) = 0.001$ (Prior for user)

We can also deduce:

- $P(T = 1|D = 0) = 1 - P(T = 0|D = 0) = 1 - 0.99 = 0.01$ (False Positive)
- $P(D = 0) = 1 - P(D = 1) = 1 - 0.001 = 0.999$ (Prior for non-user)

The only unknown is the denominator $P(T = 1)$, which is the total probability of testing positive. We find this using the law of total probability:

$$P(T = 1) = P(T = 1|D = 1)P(D = 1) + P(T = 1|D = 0)P(D = 0)$$

Plugging in the numbers:

$$\begin{aligned} P(T = 1) &= (0.99 \times 0.001) + (0.01 \times 0.999) \\ &= 0.00099 + 0.00999 \\ &= 0.01098 \end{aligned}$$

(This matches the 0.01098 in the code's output for $P(T = 1)$.)

Now we can solve the original Bayes' formula:

$$P(D = 1|T = 1) = \frac{P(T = 1|D = 1)P(D = 1)}{P(T = 1)} = \frac{0.00099}{0.01098} \approx 0.0901639$$

So, there is approximately a 9% chance that a person who tests positive is actually a drug user. This result, often surprisingly low, is known as the base rate fallacy.

7.1.2 Python Implementation

The Python code below calculates these same values using matrix operations. p_{t_d} is the conditional probability matrix $P(T|D)$, and p_d is the prior $P(D)$. The joint probability $P(T, D)$ is calculated, and then the conditional $P(D|T)$ is found by normalizing the joint.

```
1  import numpy as np
2  p_t1_d1 = 0.99
3  p_t0_d0 = 0.99
4  p_t_d = np.array([[p_t0_d0, 1-p_t0_d0], [1-p_t1_d1, p_t1_d1]])
5  p_d1 = 0.001
6  p_d = np.array([1-p_d1, p_d1])
7  p_t = p_t_d.T @ p_d
8  print("Probability of T:", p_t)
9  print("Probability of D:", p_d)
10 p_joint = p_t_d * p_d.reshape(-1, 1)
11 print("Joint Probability (P(T, D)):", p_joint)
12 p_d_t = p_joint / p_t
13 print("Conditional Probability (P(D|T)):", p_d_t)
14 print("P(D=1|T=1):", p_d_t[1][1])
```

7.1.3 Program Output

```
Probability of T: [0.98902 0.01098]
Probability of D: [0.999 0.001]
Joint Probability (P(T, D)): [[9.8901e-01 9.9900e-03]
[1.0000e-05 9.9000e-04]]
Conditional Probability (P(D|T)): [[9.99989889e-01 9.09836066e-01]
[1.01110190e-05 9.01639344e-02]]
P(D=1|T=1): 0.09016393442622944
```

7.2 Part 2

a

We have 10 total training examples.

- $P(y = 1)$: We count the number of times $y = 1$ appears in the $\mathbf{y}$ vector.
 - $\text{Count}(y = 1) = 5$
 - $P(y = 1) = \frac{\text{Count}(y=1)}{\text{Total Examples}} = \frac{5}{10} = 0.5$
- $P(y = 0)$: We count the number of times $y = 0$ appears in the $\mathbf{y}$ vector.

- $\text{Count}(y = 0) = 5$
- $P(y = 0) = \frac{\text{Count}(y=0)}{\text{Total Examples}} = \frac{5}{10} = 0.5$

(b) Compute the estimates of the 4 conditional probabilities:

First, let's subset the data based on the class label.

- **Data for $y = 1$ (5 examples):**

[0, 1]
[1, 1]
[1, 1]
[1, 1]
[1, 1]

- **Data for $y = 0$ (5 examples):**

[0, 0]
[0, 0]
[1, 0]
[1, 0]
[1, 0]

Now, we compute the required probabilities (for the test example $\mathbf{x} = [1, 1]$) based on these subsets.

- $P(x_1 = 1|y = 1)$:
 - Among the 5 examples where $y = 1$, we count how many have $x_1 = 1$.
 - Count = 4
 - $P(x_1 = 1|y = 1) = \frac{4}{5} = 0.8$
- $P(x_2 = 1|y = 1)$:
 - Among the 5 examples where $y = 1$, we count how many have $x_2 = 1$.
 - Count = 5
 - $P(x_2 = 1|y = 1) = \frac{5}{5} = 1.0$
- $P(x_1 = 1|y = 0)$:
 - Among the 5 examples where $y = 0$, we count how many have $x_1 = 1$.
 - Count = 3
 - $P(x_1 = 1|y = 0) = \frac{3}{5} = 0.6$
- $P(x_2 = 1|y = 0)$:
 - Among the 5 examples where $y = 0$, we count how many have $x_2 = 1$.
 - Count = 0
 - $P(x_2 = 1|y = 0) = \frac{0}{5} = 0.0$

(c) Compute the most likely label for the test example:

We need to classify the test example $\mathbf{x} = [1, 1]$. We use the Naive Bayes assumption to compare the (unnormalized) posterior probabilities, or "scores," for each class.

$$\text{Score}(y) \propto P(y) \times P(x_1|y) \times P(x_2|y)$$

- **Score for class $y = 1$:**

$$\begin{aligned}\text{Score}(y = 1) &\propto P(y = 1) \times P(x_1 = 1|y = 1) \times P(x_2 = 1|y = 1) \\ &= 0.5 \times 0.8 \times 1.0 \\ &= 0.4\end{aligned}$$

- **Score for class $y = 0$:**

$$\begin{aligned}\text{Score}(y = 0) &\propto P(y = 0) \times P(x_1 = 1|y = 0) \times P(x_2 = 1|y = 0) \\ &= 0.5 \times 0.6 \times 0.0 \\ &= 0.0\end{aligned}$$

Conclusion: Since $\text{Score}(y = 1) > \text{Score}(y = 0)$ ($0.4 > 0.0$), the Naive Bayes model predicts the most likely label for the test example $\mathbf{x} = [1, 1]$ is **1**.

Python Code

```
import numpy as np

X = np.array([
    [0, 1],
    [1, 1],
    [0, 0],
    [1, 1],
    [1, 1],
    [0, 0],
    [1, 0],
    [1, 0],
    [1, 1],
    [1, 0]
])

y = np.array([1, 1, 0, 1, 1, 0, 0, 0, 1, 0])
p_y = np.array([len(y[y == 0]), len(y[y == 1])]) / len(y)
p_y

array([0.5, 0.5])
```

```

X_0 = X[y == 0]
X_1 = X[y == 1]
p_x_given_y0 = X_0.sum(axis=0) / len(y[y == 0])
p_x_given_y1 = X_1.sum(axis=0) / len(y[y == 1])
p_x_1_given_y = np.array([p_x_given_y0, p_x_given_y1])
p_x_0_given_y = 1 - p_x_1_given_y
p_x_given_y = np.stack([p_x_0_given_y, p_x_1_given_y])
p_x_given_y

array([[0.4, 1. ],
       [0.2, 0. ]],

       [[0.6, 0. ],
        [0.8, 1. ]]])

def calculate_prob(x, p_x_given_y, p_y):

    prob_x1_y0 = p_x_given_y[x[0], 0, 0]
    prob_x2_y0 = p_x_given_y[x[1], 0, 1]
    score_0 = p_y[0] * prob_x1_y0 * prob_x2_y0

    prob_x1_y1 = p_x_given_y[x[0], 1, 0]
    prob_x2_y1 = p_x_given_y[x[1], 1, 1]
    score_1 = p_y[1] * prob_x1_y1 * prob_x2_y1
    print("probability of class 0:", score_0)
    print("probability of class 1:", score_1)
    if (score_1 > score_0):
        return 1
    else:
        return 0

test_x = np.array([1, 1])
prediction = calculate_prob(test_x, p_x_given_y, p_y)
print(f"Input: {test_x}, Prediction: {prediction}")

probability of class 0: 0.0
probability of class 1: 0.4
Input: [1 1], Prediction: 1

```